CASE STUDY REPORT

SKILL HUB

An ML-Based Student Career Readiness Prediction and Adaptive Skill-Gap Analysis Platform

Submitted in fulfillment of the requirements for the subject
MACHINE LEARNING FOR DATA SCIENCE

Submitted by: Vigasni R , Ananya H Narasimha , Shalini V K

Register Number: 24MID0020 , 24MID0174 , 24MID0308

School / Programme: SCOPE / MID

Institution: VIT Vellore

Submitted to: Dr . Sudhakara Pandian R

Academic Year: 2026–2027

Abstract

Online learning platforms commonly organize learning around courses, videos, quizzes, and completion records. However, course completion alone does not necessarily prove that a learner has acquired the competencies required for a chosen career. This case study proposes Skill Hub, an integrated machine-learning and adaptive-learning platform that combines student career-readiness prediction with skill-gap analysis and personalized learning-path generation. A student selects a target such as Data Scientist, completes diagnostic and practical assessments, and receives estimated skill mastery and an overall readiness category. The system compares current mastery with target requirements, represents prerequisite relationships through a directed skill graph, and prioritizes gaps according to gap size, prerequisite importance, career relevance, and recent performance trends. Recommendations are updated as the learner completes learning activities and reassessments.

The proposed machine-learning component may be implemented as a supervised classification model to predict readiness categories (for example, developing, nearly ready, or ready), and/or as a regression model to estimate a continuous readiness score. Candidate algorithms include Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting, selected through validation rather than assumed to be best in advance. The reference paper supplied with the project is used to inform the academic structure and discuss optimization of neural-network connectivity; its reported experiments are not presented as results of Skill Hub. No project dataset, implementation logs, or measured evaluation results were supplied. Accordingly, this report distinguishes the proposed methodology and illustrative examples from experimentally verified findings and provides a reproducible evaluation plan for future testing.

Keywords: Machine Learning, Career Readiness Prediction, Skill-Gap Analysis, Adaptive Learning, Prerequisite Graph, Personalized Recommendation, Student Analytics.

Contents

- 1. Introduction
- 2. Problem Statement and Objectives
- 3. Related Work and Reference Study
- 4. Proposed Skill Hub Framework
- 5. Machine-Learning Methodology
- 6. System Design and Implementation Plan
- 7. Evaluation Plan and Performance Metrics
- 8. Illustrative Example (Not Experimental Results)
- 9. Discussion, Limitations and Ethical Considerations
- 10. Conclusion and Future Work
- References
- Appendix A: Proposed Algorithm
- Appendix B: Suggested Dataset Schema

1. Introduction

1.1 Background

Machine learning can help educational systems analyze assessment evidence, estimate learning progress, and support decisions about what a student should learn next. A student's readiness for a career is not determined by a single course-completion percentage. It depends on a set of related competencies, such as programming, statistics, databases, data analysis, communication, and practical problem-solving. These competencies may have prerequisite relationships: for example, probability and statistics support later learning in machine learning.

Skill Hub combines two connected functions. First, a readiness-prediction component estimates a learner's current readiness for a selected career goal using assessment and progress data. Second, an adaptive skill engine identifies individual gaps and recommends a learning sequence that respects prerequisite relationships. The system is intended to update its recommendations after each new assessment, rather than treating the initial result as permanent.

1.2 Motivation

The project idea was motivated by the limitations of course-based learning systems that may report progress without establishing whether a skill has been demonstrated. Skill Hub therefore focuses on evidence of competency. It uses diagnostic tests, practice results, attempts, time taken where appropriate, and practical-task performance to estimate mastery. The platform can then prioritize foundational gaps that block progress toward a career goal.

1.3 Scope

- Provide a student profile and target-career selection.
- Store career requirements, skills, prerequisite links, assessments, and learning resources.
- Estimate skill mastery from assessment evidence.
- Predict a career-readiness score or category using a supervised machine-learning model, once suitable labelled data are available.
- Rank skill gaps and generate a personalized, prerequisite-aware learning path.
- Reassess learners and update their profile and recommendations.
- Support practical skill verification as a proposed extension.

2. Problem Statement and Objectives

2.1 Problem Statement

Many learning platforms present the same course sequence to different learners and rely heavily on course completion or quiz scores. These signals may not identify which prerequisite is missing, how strongly that missing skill affects career readiness, or what the learner should study next. A student targeting a Data Scientist role may have high Python proficiency but weak probability and machine-learning knowledge. Recommending an advanced machine-learning course immediately could be ineffective if foundational concepts are not secure.

The problem addressed in this case study is to design a system that estimates student career readiness from observable evidence, detects weighted skill gaps, uses a dependency graph to identify prerequisite deficiencies, and adapts the learning path as new evidence becomes available.

2.2 Objectives

1. Design an integrated framework for career-readiness prediction and skill-gap analysis.
2. Represent skills and prerequisite relationships as a directed graph.
3. Define a transparent skill-mastery and skill-gap calculation.
4. Propose supervised ML models for readiness prediction and a fair model-selection procedure.
5. Create a learning-priority score that combines gap size, prerequisite importance, career relevance, and performance trend.
6. Specify a continuous assessment–recommendation–reassessment feedback loop.
7. Define evaluation metrics and a validation protocol without claiming results that have not been measured.

2.3 Expected Contribution

The intended contribution is the integration of a readiness estimate with a graph-aware recommendation mechanism. Instead of treating the model's predicted category as the only decision, Skill Hub combines the prediction with explainable skill-level evidence. This is a proposed system design, not a claim that the system has already achieved a measured improvement.

3. Related Work and Reference Study

3.1 Artificial Neural Networks and Optimization

The supplied reference article by Yılmaz and Kıran (2026), "Connectivity Optimization of Artificial Neural Network with Evolutionary Computation Algorithms," studies a neural network whose connectivity structure and weights are optimized separately. The authors use Binary Particle Swarm Optimization (BPSO) to determine whether connections should exist and continuous Particle Swarm Optimization (PSO) to optimize the weights of active connections. They compare the approach with neural-network baselines and a genetic-algorithm-based connectivity method on four benchmark datasets: Breast Cancer, Hepatitis, Mammographic Mass, and Haberman's Survival.

The article explains that connection pruning can reduce unnecessary inputs and hidden neurons, potentially producing a simpler network. It reports results using accuracy, recall, specificity, precision, and F1-score, with 10-fold cross-validation. These details are relevant as background for thinking about model complexity and evaluation. However, the paper does not evaluate Skill Hub, student career readiness, or the project's proposed priority engine. Its numerical results therefore must not be attributed to this project.

3.2 Relevance to Skill Hub

The reference study is useful in three ways: it demonstrates that model structure and parameters can be treated as optimization problems; it emphasizes the importance of comparing multiple methods using consistent evaluation metrics; and it illustrates the value of reporting variability rather than only a single score. Skill Hub adopts these general evaluation principles, but its primary proposed task is supervised prediction using student competency evidence combined with a graph-based recommendation layer.

3.3 Proposed Positioning

The proposed system differs from a conventional neural-network connectivity-optimization study. Its focus is the joint interpretation of career goal requirements, per-skill mastery, prerequisite dependencies, and changes in assessment performance over time. A neural network or evolutionary optimizer could be explored later, but neither is required for the first implementation. Simpler baseline models should be evaluated first to establish whether more complex models provide a meaningful improvement.

4. Proposed Skill Hub Framework

4.1 Core Modules

Module	Purpose	Main inputs / outputs
User and goal management	Maintain learner profile and target career.	Profile, target role, target skill requirements
Skill and career database	Store skills, required levels, dependencies, and resources.	Skill catalogue, directed prerequisite graph
Assessment engine	Deliver diagnostic, topic-based, and practical assessments.	Responses, scores, attempts, time, practical rubric
Mastery estimator	Convert assessment evidence into per-skill mastery estimates.	Mastery score for each skill
Readiness prediction	Estimate overall readiness category or score.	Feature vector → predicted class / score
Gap and priority engine	Compare mastery with requirements and rank gaps.	Gap, prerequisite importance, priority score
Adaptive learning path	Recommend the next skill and learning resources.	Ordered skill/resource recommendations
Progress and verification	Track reassessments and evidence of competence.	Updated profile, verified-skill status

4.2 Skill Dependency Graph

A skill graph can be represented as G = (V, E), where V is the set of skills and E contains directed prerequisite relationships. An edge A → B means that competence in A supports learning B. The graph should be reviewed by subject experts and may contain weighted edges to represent the strength of each dependency. A graph for a Data Science goal could include Python → Pandas/NumPy → Data Analysis, Statistics → Probability → Machine Learning, and SQL → Data Analysis.

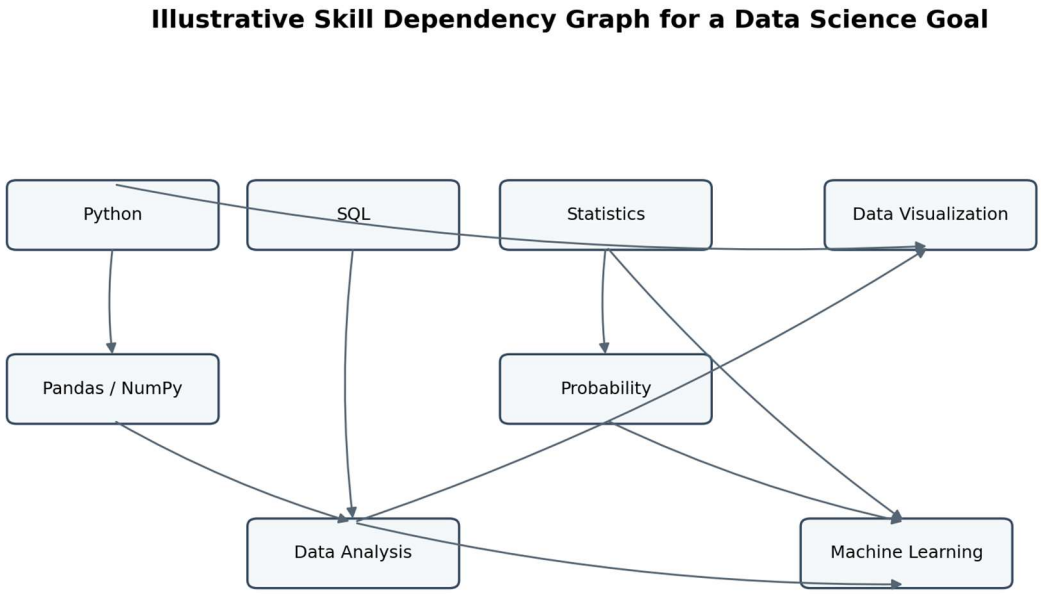


Figure 1. Illustrative prerequisite graph. Relationships are proposed examples and should be validated for the target curriculum.

4.3 Skill Mastery and Skill Gap

For each skill s, the platform stores a current mastery estimate M_s between 0 and 100 and a required level R_s between 0 and 100 for the selected career goal. A simple gap is:

$$\mathbf{Gap_s = max(0, R_s - M_s)}$$

For example, if a skill's required level is 75 and the current mastery estimate is 43, the gap is 32 points. The numbers in this example are illustrative, not observed results. Mastery may initially be based on normalized assessment scores; a later version could combine quiz accuracy, question difficulty, attempts, recency, and practical-task rubrics.

4.4 Learning Priority and Skill Debt

The project notes propose a concept called Skill Debt: important knowledge gaps that may prevent progress toward a goal. To operationalize this concept, the system can calculate a priority score using normalized values:

$$\mathbf{Priority_s = Gap_s \times PrerequisiteWeight_s \times GoalRelevance_s \times TrendFactor_s}$$

Each factor should be normalized to a documented range before multiplication. Gap represents the size of the missing competency; PrerequisiteWeight represents how strongly the skill supports downstream skills; GoalRelevance represents its importance for the selected career; and TrendFactor increases or decreases priority according to recent performance. The final score can be normalized to 0–100 for display. The exact factor ranges and weights are design choices that must be tuned and evaluated, not assumed to be optimal.

A skill with a moderate gap may receive higher priority than a skill with a larger gap if it is a prerequisite for several important target skills. This graph-aware ranking is the main reason the proposed platform is more informative than simply recommending the weakest skill.

4.5 Career Readiness Prediction

The ML component estimates readiness from an input vector assembled from skill mastery, assessment history, practical performance, completion consistency, and other features that are available and appropriate. Two output formats are possible:

- Classification: predict a readiness category such as Developing, Nearly Ready, or Ready.
- Regression: predict a continuous readiness score between 0 and 100.

The target labels must have a defensible definition. For example, categories could be created using an expert-reviewed rubric or a validated practical assessment. If labels are derived directly from the same skill scores used as input, the model may merely reproduce the rule rather than learn an independently validated concept. The project should document how labels are generated.

4.6 Continuous Adaptation

After a learner studies a recommended topic, the system administers a reassessment. The mastery estimate is updated, gaps and priorities are recalculated, and the next learning path is regenerated. A skill should be marked as verified only after meeting a defined evidence threshold, such as a required score on a fresh assessment and a satisfactory practical task. Course completion alone should not automatically imply verified mastery.

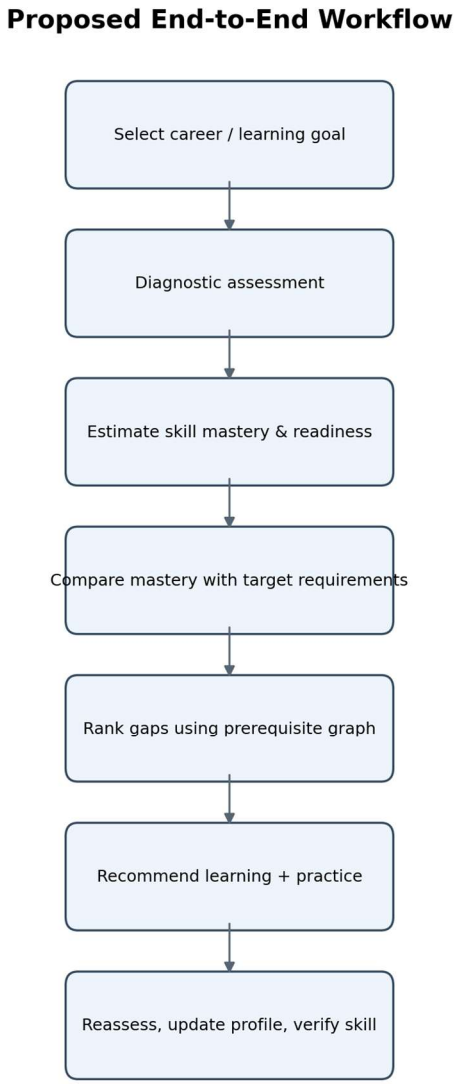


Figure 2. Proposed workflow from goal selection to reassessment.

5. Machine-Learning Methodology

5.1 Problem Formulation

The prediction task is supervised learning if each training example has a target readiness label or score. Let X denote the feature vector for a student at a particular assessment time and y denote the readiness target. A classifier learns f(X) → y_class; a regressor learns f(X) → y_score. The prediction is then used alongside the skill graph and mastery-gap calculations to support recommendations.

5.2 Candidate Features

Feature group	Example variables	Reason for inclusion
Skill mastery	Python, SQL, statistics, probability, data analysis, ML mastery	Represents current competency evidence
Assessment performance	Accuracy, score by difficulty, attempts, recent assessment results	Measures demonstrated knowledge
Practical performance	Rubric score, task completion, code-test pass rate	Adds applied evidence beyond MCQs
Learning trend	Change in scores across recent assessments	May indicate improvement or decay
Learning activity	Practice completion, revision frequency, time-on-task where	Provides context about engagement

	appropriate	
Goal requirements	Required skill levels and career-specific relevance	Connects the profile to the selected target

Features should be available at prediction time and collected consistently. Direct identifiers such as name or email should not be used as predictive features. Features that could introduce unfairness or proxy sensitive characteristics should be excluded or carefully justified.

5.3 Candidate Algorithms

Algorithm	Strengths	Limitations / role
Logistic Regression	Interpretable baseline for categorical readiness.	May underfit nonlinear relationships.
Decision Tree	Readable decision rules; handles nonlinear splits.	Can overfit without depth/pruning controls.
Random Forest	Captures nonlinear patterns and interactions; robust baseline.	Less directly interpretable than a single tree.
Gradient Boosting	Often effective on structured/tabular data.	Requires tuning and careful validation.
Neural Network	Can model complex relationships with sufficient data.	Needs more data/tuning; may be harder to explain.

These are candidate methods, not a declaration of the best model. The final choice should be based on validation performance, calibration where relevant, interpretability, data volume, and fairness checks. A simple rule-based readiness baseline should also be included so the ML model must demonstrate value over a transparent alternative.

5.4 Data Preprocessing

8. Remove duplicate records and inspect missing values.
9. Encode categorical variables using a method appropriate to the model.
10. Scale numeric features when required, especially for distance- or gradient-sensitive models.
11. Split data by student, not just by individual assessment row, when the same student contributes multiple records; this reduces leakage between training and test data.
12. Fit imputers, scalers, and encoders only on the training portion of each fold.
13. Address class imbalance only inside the training folds, and compare class-weighting or resampling approaches without contaminating the test data.
14. Document all feature definitions, label rules, exclusions, and transformations.

5.5 Training and Validation Protocol

A proposed evaluation protocol is to reserve a final test set and use stratified cross-validation on the remaining training data for classification. If records are time-ordered, a time-based split may better represent future deployment. If students have repeated assessments, group-based splitting by student is preferable. Hyperparameters should be selected using only training/validation data. The final test set should be evaluated once after model selection.

The supplied reference paper used a 90%/10% train-test division and 10-fold cross-validation on its training data. Skill Hub may use a similar general structure if the eventual dataset size and dependence structure make it appropriate; the split must be chosen based on the actual data rather than copied automatically.

5.6 Recommendation Engine

The recommendation engine is a separate decision layer rather than simply the output of the classifier. A proposed sequence is:

15. Estimate mastery for each target skill.
16. Calculate the gap between mastery and the required level.
17. Identify unmet prerequisites for target skills in the dependency graph.

18. Compute a normalized priority score for each eligible skill.
19. Exclude or defer skills whose prerequisites are not sufficiently mastered, unless a diagnostic activity is recommended first.
20. Recommend the highest-priority eligible skill and suitable learning resources.
21. Reassess after learning and recalculate the recommendation.

5.7 Proposed End-to-End Algorithm

22. Input: target career, required skill levels, skill dependency graph, learner assessment history.
23. Preprocess assessment evidence and estimate mastery M_s for each skill s.
24. Construct the feature vector X from valid, available learner evidence.
25. Use the selected trained model to predict readiness class or score.
26. For each target skill, calculate $Gap_s = \max(0, R_s - M_s)$.
27. Use graph dependencies, goal relevance, and performance trend to compute $Priority_s$.
28. Rank eligible skills and select the next skill or prerequisite remediation task.
29. Present the recommendation with an explanation of the gap and downstream skills it supports.
30. After learning, collect new assessment evidence, update mastery and trend, and repeat.

6. System Design and Implementation Plan

6.1 Proposed Architecture

The platform can be implemented as a web application with a frontend, REST API, prediction service, graph-aware recommendation engine, and database. The technology choices below are drawn from the supplied project notes and represent a suggested implementation rather than a confirmed deployed stack.

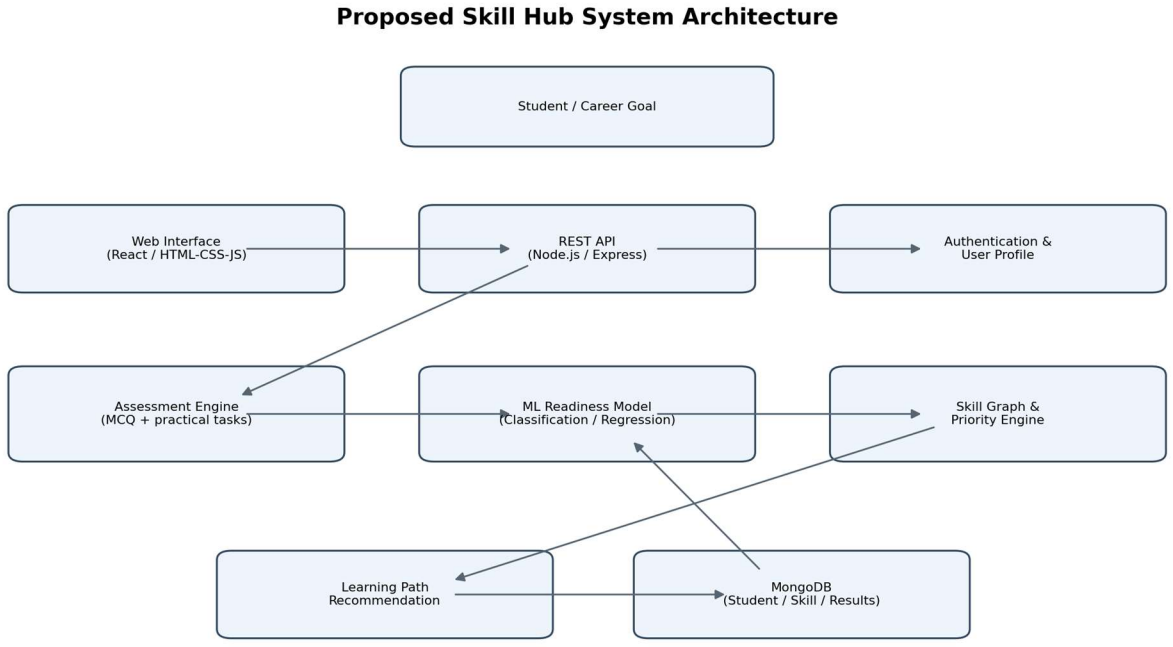


Figure 3. Proposed software and ML architecture.

6.2 Suggested Technology Stack

Layer	Suggested technology	Responsibility
Frontend	React.js or HTML, CSS, JavaScript	Dashboard, assessments, skill map, recommendations
Backend	Node.js and Express.js	API routes, validation, authentication, business logic
Database	MongoDB	Profiles, skills, prerequisites, questions, scores, resources
ML service	Python with scikit-learn	Preprocessing, training, evaluation, prediction
Visualizations	Chart.js or equivalent	Mastery progress, skill gaps, readiness history
Security	JWT-based sessions, password hashing, access controls	Protect profiles and assessment records
Versioning	Git / GitHub	Track code, model versions, and documentation

6.3 Suggested Database Collections

Collection	Example fields
Users	user_id, display_name, email, hashed_password, selected_goal
Careers	career_id, title, required_skill_ids
Skills	skill_id, name, category, difficulty
SkillPrerequisites	skill_id, prerequisite_id, dependency_weight
Questions	question_id, skill_id, difficulty, question, options, answer_key
Assessments	assessment_id, user_id, skill_id, score, attempts, time_taken, date
SkillProgress	user_id, skill_id, mastery_score, required_score, gap, trend
LearningResources	resource_id, skill_id, title, type, url, difficulty
ReadinessPredictions	user_id, model_version, prediction, confidence_or_score, created_at

6.4 User Journey

31. The student registers and selects a target role, such as Data Scientist.
32. The platform loads the role's required skills and prerequisite graph.
33. The student completes a diagnostic assessment and any available practical tasks.
34. The system calculates skill mastery and predicts a readiness category or score.
35. The gap engine identifies missing competencies and ranks prerequisite-sensitive priorities.
36. The dashboard explains the next recommended skill and why it matters.
37. The student uses learning materials and practice activities, then completes a reassessment.
38. The system updates the skill profile and learning path; skills are marked verified only when the evidence rule is met.

7. Evaluation Plan and Performance Metrics

7.1 Prediction Metrics

Metric	Definition / use	Interpretation
Accuracy	Correct predictions divided by all predictions.	Useful overall, but may mislead with imbalanced classes.
Precision	TP / (TP + FP).	Of students predicted ready, how many meet the reference label?
Recall / Sensitivity	TP / (TP + FN).	How many truly ready (or not-ready, depending on positive class) cases are identified?
F1-score	Harmonic mean of precision and recall.	Balances precision and recall.
Confusion matrix	Counts correct and incorrect class assignments.	Shows which readiness categories are confused.
ROC-AUC / PR-AUC	Ranking quality for binary or one-vs-rest classification.	Use only when appropriate to label structure and class balance.
MAE / RMSE	Regression error for a continuous readiness score.	Lower values indicate smaller prediction errors.

7.2 Recommendation Metrics

- Prerequisite validity: proportion of recommendations whose required prerequisites are satisfied or explicitly assigned as remediation.
- Top-k relevance: whether recommended skills align with expert-defined needs or subsequent successful learning.
- Gap reduction: change in the gap score between pre-learning and post-learning assessments.
- Learning gain: change in skill-specific assessment performance after a learning intervention.
- Time to mastery: time or number of attempts required to meet a pre-defined mastery threshold.
- User feedback: perceived usefulness, clarity, and relevance of recommendation explanations.

7.3 Baselines and Comparisons

Compare the ML readiness model with at least one simple baseline, such as a rule-based score derived from weighted skill mastery. Compare recommendation ranking against a weakest-skill-first strategy and a career-relevance-only strategy. Use the same held-out evaluation cases and report means and variability where repeated folds or runs are used. Avoid claiming the proposed method is superior until such comparisons have been conducted.

7.4 Experimental Status

Important status note: No Skill Hub dataset, trained model, code execution output, user study, or measured evaluation table was included in the supplied materials. Therefore, this report does not claim that the project has achieved any specific accuracy, F1-score, learning gain, or reduction in skill gaps. The methodology and metrics in this section are proposed evaluation steps. Results should be added only after experiments are run and documented.

8. Illustrative Example (Not Experimental Results)

The following example demonstrates how the proposed logic could operate. The values are hypothetical and are included only to explain the calculations.

Skill	Illustrative current mastery	Illustrative required level	Gap
Python	82	80	0
SQL	70	75	5
Statistics	45	75	30
Probability	32	70	38
Machine Learning	20	70	50
Data Visualization	60	70	10

Although Machine Learning has the largest raw gap in this example, the graph may identify Probability and Statistics as foundational prerequisites. If those dependencies are confirmed in the selected curriculum, the recommendation engine may prioritize Probability before recommending advanced Machine Learning content. The actual priority order depends on the defined edge weights, goal relevance, mastery thresholds, and trend factors.

A hypothetical cycle would be: (1) diagnostic test identifies low probability mastery; (2) the system recommends probability basics and practice; (3) a reassessment shows an improved score; (4) mastery and gap are updated; (5) the engine re-evaluates Statistics as the next priority. These steps illustrate the intended

feedback loop and are not evidence that the software has been implemented or that a learner achieved these values.

9. Discussion, Limitations and Ethical Considerations

9.1 Potential Benefits

- Personalization: students can receive different learning paths based on their own evidence.
- Explainability: recommendations can cite the gap, prerequisite, and career requirement that caused a skill to be prioritized.
- Continuous adaptation: new assessments can change mastery estimates and recommendations.
- Practical verification: competence can be supported by practical tasks rather than course completion alone.
- Model comparison: baseline and ML approaches can be compared using consistent metrics.

9.2 Limitations

- Prediction quality depends on the amount, quality, representativeness, and labelling of student data.
- Career readiness is multidimensional; an assessment score cannot capture every workplace competency.
- Incorrect prerequisite links or career requirement weights can produce poor recommendations.
- Repeated assessments from the same student can create data leakage if splitting is not performed carefully.
- Class imbalance may cause a model to perform poorly for less common readiness categories.
- Learning trend and skill-decay estimates require repeated measurements over time.
- Practical verification requires suitable rubrics, test cases, and reliable scoring.

9.3 Fairness, Privacy and Responsible Use

Student assessment records are sensitive educational data. The system should collect only necessary information, restrict access, protect stored credentials, and provide a clear explanation of how predictions are used. Readiness predictions should support learning rather than make high-stakes decisions automatically. Evaluation should check performance across relevant student groups where lawful and appropriate, and the platform should avoid using protected or irrelevant personal attributes. Students should be able to challenge inaccurate records and retake assessments when appropriate.

9.4 Future Extensions

- Add a skill-decay detector using time-stamped assessment history.
- Use a calibrated readiness probability and show uncertainty rather than only a hard label.
- Explore graph neural networks or knowledge-tracing methods only when sufficient sequential data are available.
- Add practical coding tasks and project-based evidence for verification.
- Conduct a pilot study with consent, predefined outcomes, and an appropriate comparison group.
- Investigate the novelty of the priority mechanism through a prior-art review before making any patentability claim.

10. Conclusion and Future Work

This case study proposes Skill Hub as an integrated machine-learning and adaptive-learning platform for student career readiness. The system combines supervised readiness prediction with skill mastery estimation, gap calculation, prerequisite graph analysis, priority ranking, personalized resource recommendations, and

reassessment. Its central idea is to recommend not merely the weakest skill, but the most useful next skill in the context of the student's chosen goal and prerequisite structure.

The supplied research paper provides background on neural-network connectivity optimization and the importance of systematic model evaluation. Its experimental findings are treated as reference-study findings only; they are not attributed to Skill Hub. Since no project-specific dataset or implementation results were supplied, the present report defines a proposed design and evaluation plan rather than reporting completed experiments. The next steps are to build a small working prototype, prepare a documented dataset with defensible readiness labels, compare baseline and candidate models using leakage-aware validation, evaluate recommendation quality, and report the observed results with appropriate limitations.

References

[1] M. Yılmaz and M. S. Kıran, "Connectivity optimization of artificial neural network with evolutionary computation algorithms," Turkish Journal of Engineering, vol. 10, no. 4, pp. 1237–1250, 2026. doi: 10.31127/tuje.1962552.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5–32, 2001.

[4] C. M. Bishop, Pattern Recognition and Machine Learning. New York: Springer, 2006.

[5] T. M. Mitchell, Machine Learning. New York: McGraw-Hill, 1997.

Appendix A: Proposed Pseudocode

```
INPUT:
    target_career
    required_level[skill]
    prerequisite_graph G
    assessment_history
    trained_readiness_model

1. For each skill s required by target_career:
       mastery[s] = estimate_mastery(assessment_history, s)
       gap[s] = max(0, required_level[s] - mastery[s])

2. features = build_prediction_features(mastery, assessment_history,
                                        target_career)
3. readiness_prediction = trained_readiness_model.predict(features)

4. For each skill s:
       prerequisite_weight = graph_dependency_weight(G, s)
       goal_relevance = career_relevance(target_career, s)
       trend_factor = estimate_performance_trend(assessment_history, s)
       priority[s] = normalize(gap[s] * prerequisite_weight *
                               goal_relevance * trend_factor)

5. eligible_skills = skills with unmet gaps and satisfied prerequisites
   If a prerequisite is not satisfied:
       recommend the prerequisite as remediation.

6. next_skill = highest_priority(eligible_skills)
```

```
7. explain recommendation using gap, prerequisite links, and career relevance
8. collect new assessment after learning
9. update mastery and repeat from Step 1

OUTPUT:
    readiness prediction
    ranked skill gaps
    next recommended learning activity
    explanation and updated progress
```

Appendix B: Suggested Dataset Schema

Field	Type / example	Notes
student_id	Pseudonymous ID	Use for grouping/splitting, not as a predictive feature.
assessment_date	Date/time	Supports chronological evaluation and trend estimation.
target_career	Categorical	The career goal selected for this assessment.
python_mastery	0–100	Estimated from assessment evidence.
sql_mastery	0–100	Estimated from assessment evidence.
statistics_mastery	0–100	Estimated from assessment evidence.
probability_mastery	0–100	Estimated from assessment evidence.
ml_mastery	0–100	Estimated from assessment evidence.
practical_score	0–100	Rubric or automated test score, where available.
attempt_count	Integer	Defined consistently per skill assessment.
recent_score_change	Numeric	Derived only from earlier assessments.
readiness_label	Category	Must have a documented, defensible labelling rule.
readiness_score	0–100	Optional regression target if a validated score exists.

Before model training, define the unit of analysis (one student at one time point, or one completed assessment), the readiness label, how missing values are handled, and how repeated records from a student are separated across training and test data.